Rollout-Training Co-Design for Efficient LLM-Based Multi-Agent Reinforcement Learning

Zhida Jiang[†], Zhaolong Xing[†], Jiawei Lu[†], Yipei Niu[§], Qingyuan Sang[†], Liangxu Zhang[†], Wenquan Dai[§], Junhua Shu[†], Jiaxing Wang[†], Qiangyu Pei[§], Qiong Chen[§], Xinyu Liu[§], Fangming Liu[*‡], Ai Han[†], Zhen Chen[*†], Ke Zhang[†]

[†]*JD.com* [§]*Huawei* [‡]*Huazhong University of Science and Technology*

arXiv:2602.09578v1 [cs.LG] 10 Feb 2026

ABSTRACT

Despite algorithm-level innovations for multi-agent reinforcement learning (MARL), the underlying networked infrastructure for large-scale MARL training remains underexplored. Existing training frameworks primarily optimize for single-agent scenarios and fail to address the unique system-level challenges of MARL, including rollout-training synchronization barriers, rollout load imbalance, and training resource underutilization. To bridge this gap, we propose FlexMARL, the first end-to-end training framework that holistically optimizes rollout, training, and their orchestration for large-scale LLM-based MARL. Specifically, FlexMARL introduces the joint orchestrator to manage data flow under the rollout-training disaggregated architecture. Building upon the experience store, a novel micro-batch driven asynchronous pipeline eliminates the synchronization barriers while providing strong consistency guarantees. Rollout engine adopts a parallel sampling scheme combined with hierarchical load balancing, which adapts to skewed inter/intra-agent request patterns. Training engine achieves on-demand hardware binding through agent-centric resource allocation. The training states of different agents are swapped via unified and location-agnostic communication. Empirical results on a large-scale production cluster demonstrate that FlexMARL achieves up to 7.3× speedup and improves hardware utilization by up to 5.6× compared to existing frameworks.

1 INTRODUCTION

Recent advances in large language models (LLMs) have facilitated the development of multi-agent systems that leverage specialized roles and distributed reasoning to solve complex tasks [14, 20, 43]. Concurrently, reinforcement learning (RL) has emerged as a powerful paradigm for improving the training performance of LLMs [10, 48]. To harness RL's policy optimization capabilities for strengthening multi-agent systems, multi-agent reinforcement learning (MARL) enables a collective of agents to learn coordinated policies through shared experiences [34, 46]. Such a learning paradigm continuously collects experience generated by multi-agent interactions, integrates feedback from reward signals, and refines model parameters to unleash collaborative intelligence [11, 50, 56].

Despite the impressive performance gains brought by MARL, most research pays more attention to algorithm-level innovations, such as communication protocol design [41, 53], coordination strategy optimization [29], and reward function refinement [40]. The underlying networked infrastructure that supports large-scale MARL training remains underexplored. The collaborative characteristics of LLM-based MARL differ fundamentally from those of general agentic RL, creating three system-level challenges that severely limit the large-scale training efficiency. *(1) Rollout-training synchronization barriers.* Some agent interactions require an extremely long query response time, i.e., long-tail effect. Policy training has to wait for the slowest rollout to complete, causing synchronization barriers. *(2) Rollout load imbalance.* Rollout request scheduling exhibits a severe imbalance among agents during parallel trajectory generation. Some core agents are repeatedly involved and incur request queuing, while auxiliary agents remain underutilized. *(3) Training resource underutilization.* MARL demonstrates dynamic resource demands during policy training phases. Different subsets of agents could be activated for gradient computation based on the collected experience. The computing and memory resources occupied by inactive agents remain idle under static allocation strategies, resulting in poor resource utilization.

Unfortunately, mainstream RL frameworks (e.g., OpenRLHF [15], veRL [32], and AReaL [7]) only support single-agent scenarios and cannot address the above inherent challenges of MARL. Although very recent frameworks represented by MARTI [50] have attempted to support MARL, they offer basic training capabilities. These studies are customized for specialized MARL algorithms and do not provide in-depth rollout and training optimization from an infrastructure perspective. As a result, synchronization barriers, rollout load imbalance, and training resource underutilization still hinder efficient MARL training. Besides, their implementation fails

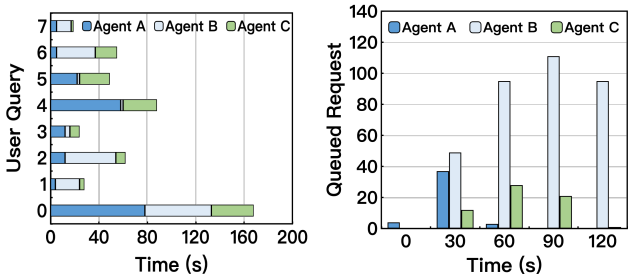
*Corresponding Authors.

to support cross-node placement for individual agents, and LLM-based agents are confined to limited parameters (e.g., 3B), preventing the exploration of large-scale MARL. The above discussions highlight the urgent need for a comprehensive infrastructure solution that optimizes the lifecycle of large-scale MARL training instead of simple integration.

To fill this gap, we propose the first end-to-end MARL training framework, termed FlexMARL, that co-designs rollout, training, and their orchestration tailored to the unique characteristics of large-scale MARL. Our framework rethinks resource allocation and data movement across the distributed networked infrastructure through three core components. *Firstly*, FlexMARL introduces a joint orchestrator based on the disaggregated architecture that coordinates rollout and training phases onto dedicated resource pools, thereby alleviating resource conflicts. The experience store serves as the foundation for structured data flow management. On this basis, we integrate a micro-batch driven asynchronous pipeline to eliminate the long-tail effect while maintaining synchronous training semantics. *Secondly*, we design a parallel sampling scheme within the rollout engine that enables concurrent generation of multi-agent interaction trajectories, thereby reducing inference latency. FlexMARL uses a manager to monitor multi-agent workloads, and then adopts hierarchical load balancing to flexibly adjust the number of inference instances through optimized D2D communication. *Thirdly*, for training optimization, we collectively manage all training processes for an individual agent. Hardware resources are dynamically allocated to agents only where and when needed, thus avoiding resource idleness. FlexMARL abstracts multi-tier pathways into a unified and location-agnostic `Set/Get` API, which are used to efficiently swap in/out the training states of suspended and resumed agents.

During each MARL step, the rollout engine generates multi-agent trajectories via parallel sampling, with inference instances elastic scaling. These trajectories are collected into the experience store, where the joint orchestrator dispatches micro batches asynchronously to the training engine. The training engine dynamically allocates resources and swaps states for policy training, and updated models are synchronized back to inference instances to start the next step. To our best knowledge, FlexMARL is the first system-level solution that holistically optimizes the rollout, training, and orchestration phases of large-scale LLM-based MARL. Overall, we make the following contributions in this paper:

- **Joint Orchestrator.** We introduce the experience store for seamless rollout-training collaboration. A micro-batch asynchronous pipeline is designed to decouple gradient computation from parameter updating and preserve synchronous training semantics.



(a) Query Response Time (b) Queued Request

Figure 1: Preliminary experiments demonstrating the MARL characteristics.

- **Rollout Engine.** We develop a parallel sampling scheme with hierarchical load balancing, which reduces trajectory generation latency and balances intra-agent/inter-agent rollout workloads.
- **Training Engine.** We exploit agent-centric resource allocation and efficient state swap mechanism to enable on-demand binding, thus adapting to dynamic resource demands and improving utilization.
- **Industrial-Scale Evaluation.** We implement and evaluate FlexMARL on a production cluster. Experimental results for real-world e-commerce multi-agent assistants show that FlexMARL provides up to 7.3× speedup and improves hardware utilization by up to 5.6× compared to existing MARL frameworks.

2 BACKGROUND AND MOTIVATION

2.1 Multi-Agent Reinforcement Learning

Traditional non-LLM MARL is largely restricted to well-structured environments with predefined action sets [2, 24, 51]. In contrast, LLM-based MARL represents a paradigm shift from low-level control optimization to knowledge-driven, high-level collaboration. The agents are instantiated as LLMs with inherent natural language understanding and reasoning capabilities. These LLM agents process unstructured information and generate token sequences as actions. As a result, LLM-based MARL effectively decomposes task complexity, enabling agents to learn collaboratively through interactions with other agents [23, 59]. Recent advancements have highlighted its effectiveness in solving long-horizon tasks such as code generation [28, 57], mathematical reasoning [21, 52], and network management [38, 42], which are challenging for conventional neural network policies.

The typical workflow of LLM-based MARL contains repeated cycles across rollout and training phases. In the rollout phase, each LLM agent generates responses based on its current policy parameters, role-specific prompts, and environmental observations [56]. Multi-agent interactions produce the trajectories that capture their reasoning processes. These

Table 1: Comprehensive comparison: FlexMARL vs. State-of-the-art RL/MARL frameworks.

Frameworks	Basic Features			End-to-End Optimization		
	Multi-Agent Support	Algorithm Decoupling	Large-Scale Deployment	Asynchronous Rollout-Training	Rollout Load Balancing	Agent-Centric Resource Allocation
General-Purpose RL Frameworks (e.g., OpenRLHF [15], veRL [32], AReaL [7])	✗	✓	✓	✓	✗	✗
Specialized MARL Frameworks (e.g., MARTI [50])	✓	✗	✗	✗	✗	✗
FlexMARL (Ours)	✓	✓	✓	✓	✓	✓

experiences are then evaluated using a rule-based or learned reward model that reflects task correctness, collaboration effectiveness, and alignment with global objectives. After careful credit assignment across multiple agents, the collected experiences are organized into batches and dispatched to corresponding agent policies for training [16]. Each agent independently optimizes its associated policy model using RL algorithms (e.g., GRPO [31]) to maximize the expected cumulative reward. The updated models are synchronized via D2D communication for the next step of rollouts.

2.2 Key Observations

Despite the advancements in MARL algorithms, the underlying challenges that hinder large-scale training efficiency remain underexplored. To reflect the unique characteristics of LLM-based MARL, we conduct extensive profiling of real-world multi-agent merchant assistant workloads. The setup of preliminary experiments is similar to that in Section 8. Three key observations are as follows.

Observation #1: MARL suffers from rollout-training synchronization barriers. Multi-agent interaction latency exhibits a pronounced long-tail distribution due to differences in task complexity and agent decision paths. As shown in Figure 1(a), most multi-agent interactions are completed in a short time, but the longest response time of user queries can reach nearly 170s, which dominates the overall experience collection time. Under synchronous training paradigms, policy updating is blocked by "stragglers" due to strict serial dependency. Policy training is triggered until all trajectories are collected, negatively affecting training throughput.

Observation #2: MARL exhibits imbalanced rollout load distributions across agents. A salient characteristic of MARL is that agents are typically assigned different roles due to task-specific collaboration mechanisms. During the experience collection phase, MARL needs to handle a large number of continuous rollout requests, exhibiting different interaction patterns based on functional roles. The rollout load among distinct agents shows a highly skewed distribution. We select three representative agents and track the number of their queued requests over time. As illustrated in Figure 1(b), a small portion of core agents are repeatedly invoked, handling over 76% of the total rollout requests, whereas other auxiliary agents process less than 24% of the requests. Such an imbalanced workload causes the core agents' requests to continuously queue, which prolongs the trajectory sampling time and becomes a bottleneck.

Observation #3: MARL demonstrates dynamic resource demands during policy training phases. The policy training phase may activate different subsets of agents for gradient computation based on the collected samples. If we allocate dedicated resources to each agent using a fixed mapping strategy, the computing and memory resources occupied by inactive agents are wasted rather than reallocated to active agents with high current demand. Our preliminary experiments also confirm that the static allocation strategy lacks a global perspective, resulting in average hardware utilization of only 18.8% during policy training.

2.3 Limitations of Existing Works

Our key observations reveal the unique characteristics of MARL, creating system-level challenges that cannot be addressed by naively extending single-agent infrastructure. As summarized in Table 1, existing RL frameworks generally fall into two categories, neither of which simultaneously addresses the challenges of synchronization barriers, rollout load imbalance, and training resource underutilization exhibited by large-scale MARL training.

General-Purpose RL Frameworks. Frameworks like OpenRLHF [15], veRL [32], and AReaL [7] have achieved impressive efficiency in LLM post-training through asynchronous pipelines [13, 58] and advanced scheduling optimizations [35, 47]. However, their solutions are primarily designed for a single model, overlooking the collaboration characteristics and complexities of MARL. For instance, in single-agent RL, all requests traverse the same LLM with homogeneous computational characteristics, whereas multiple agents with functionally specialized roles exhibit heterogeneous resource demands in MARL. Their scheduling mechanisms treat all requests uniformly, leading to imbalanced agent loads in multi-agent scenarios. Furthermore, these frameworks lack flexible resource allocation strategies that cannot adapt to dynamic demands during MARL training [49]. As a result, substantial resource underutilization persists when applied directly to MARL settings.

Specialized MARL Frameworks. Recent prototypes [50, 56] have provided basic MARL training capabilities by enabling agent interactions and communication protocols. Although these frameworks demonstrate promising algorithmic contributions, they treat the underlying networked infrastructure as a black box without end-to-end optimization [17]. Serialized dependency under synchronous paradigms further exacerbates long-tail latency. The adopted colocated architecture also forces the rollout and training to share the same resource pool. These studies rely on static resource allocation strategies that bind dedicated hardware to different agents during both rollout and training phases. Therefore, skewed rollout workloads and dynamic training demands result in poor resource utilization, thus degrading training efficiency. More importantly, their implementations cannot manage the complex resource placement across nodes and are confined to small-scale deployments with limited model parameters [4]. For instance, MARTI [50] is deployed on a 3-node cluster, and the largest model participating in multi-agent training is Qwen2.5-3B. Such constrained configurations prevent the exploration of performance gains from larger agent ensembles and more powerful models, thereby limiting scalability.

In summary, existing frameworks either lack native multi-agent support or fail to deliver end-to-end systematic optimization, thus hindering the efficient training of large-scale MARL. This reveals fundamental gaps between algorithmic requirements and existing infrastructure support. We are motivated to design a holistic and universal infrastructure solution tailored to the unique collaboration patterns of MARL.

3 SYSTEM OVERVIEW

We propose FlexMARL, an end-to-end framework that co-designs the rollout, training, and orchestration phases for large-scale LLM-based MARL. As illustrated in Figure 2, FlexMARL overcomes system-level challenges of synchronization barriers, rollout load imbalance, and training resource underutilization through three core components.

Joint Orchestrator (§4). Unlike the colocated architectures in existing MARL frameworks, FlexMARL adopts the disaggregated architecture that allocates dedicated resource pools for rollout and training, which avoids resource conflicts and allows independent scaling according to phase characteristics. To efficiently manage the data flow between the two phases, we introduce the *experience store*, a structured storage module that maintains globally unique, typed sample records with locality-aware data placement. Building upon the experience store, our asynchronous mechanism overlaps rollout and training at the micro-batch granularity. By decoupling gradient computation from parameter updates, we eliminate the long-tail effect of trajectory generation while preserving synchronous training semantics.

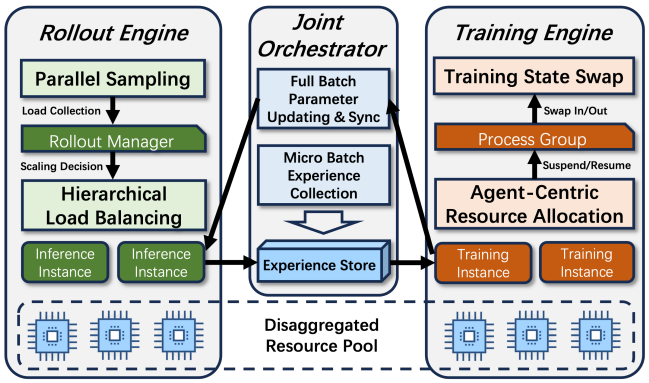


Figure 2: Overview of FlexMARL.

Rollout Engine (§5). The rollout phase in MARL is dominated by multi-agent interaction trajectories that exhibit highly skewed workloads. To improve inference efficiency, our engine employs a dependency-driven parallel sampling scheme that enables concurrent generation of inter-query and intra-query trajectories. Besides, the *rollout manager* monitors per-agent request queues in real time. Hierarchical load balancing flexibly adjusts the number of inference instances via D2D weight transmission, which relieves queuing delays and balances intra-agent/inter-agent load.

Training Engine (§6). Conventional static allocation strategy leads to severe resource waste in MARL scenarios. To maximize hardware utilization, FlexMARL features agent-centric resource allocation and manages training processes collectively through *process group*. When an agent requires policy updates, its training processes are dynamically scheduled onto available resources. We further implement fast state swap mechanisms that efficiently transfer training states (e.g., weights and optimizer states) between device and host memory. As a result, hardware resources can be flexibly allocated during active policy training, enabling FlexMARL to adapt to large-scale scenarios where a large number of agents need to be updated.

The overall workflow of FlexMARL integrates these components into a seamless loop. The rollout engine efficiently generates multi-agent interactions through parallel sampling and elastic scaling. Generated trajectories are collected in the experience store with version tracking. The joint orchestrator dispatches micro batches of experiences to the training engine with ongoing rollouts. On-demand resource binding and state swaps enable policy optimization of multiple active agents. After processing a full batch, accumulated gradients are aggregated for unified parameter updates. Finally, new policy models are synchronized to inference instances via high-bandwidth D2D interconnects.

4 JOINT ORCHESTRATOR

4.1 Rollout-Training Disaggregation

Existing training frameworks for LLM-based MARL typically employ the colocated architecture, where the rollout and training phases share the identical resource pool. However, the resource coupling design severely limits flexibility and scalability in large-scale MARL deployments. In fact, the rollout and training phases have different task characteristics. The rollout phase is memory-bound, as multiple agents generate tokens in an auto-regressive manner and manage large-scale key-value caches. In contrast, the training phase involves forward/backward propagation and gradient updating, and its performance is limited by computational power. It is suboptimal to allocate the same resources to two distinct phases, leading to resource underutilization. Beyond resource conflicts, the colocated architecture also degrades MARL's training efficiency. We have to alternate between the two phases due to time-division multiplexing of resources, incurring considerable onload/offload overhead. The frequent I/O operations result in significant communication overhead and slow down the MARL training process.

In light of the above discussion, FlexMARL advocates the disaggregated architecture for MARL. Our framework allocates dedicated, physically separate resource pools for the rollout and training phases. This decoupling design eliminates expensive I/O communication overhead, allowing each phase to fully utilize its memory and computing resources without conflict. Each phase can independently scale resources based on MARL's workload characteristics. The rollout phase can deploy more inference instances to handle concurrent multi-agent interactions without being limited by training resource requirements, thus accommodating the asymmetry of MARL workloads. Rollout-training disaggregation fundamentally resolves resource competition and offers excellent scalability.

4.2 Experience Store

While the disaggregated architecture resolves resource conflicts, it is unclear how to efficiently manage the data flow between rollout and training phases. To this end, we introduce the experience store in the joint orchestrator that enables seamless producer-consumer collaboration. Figure 3 illustrates the overall table-based data structure of the experience store, which supports efficient storage, transfer, and retrieval of training samples throughout the MARL workflow. The experience store employs a customized multi-table organization, providing clear abstractions and composability for multi-agent scenarios. Each agent is assigned to a dedicated table, which includes three categories of columns: meta-information columns, data columns, and status columns.

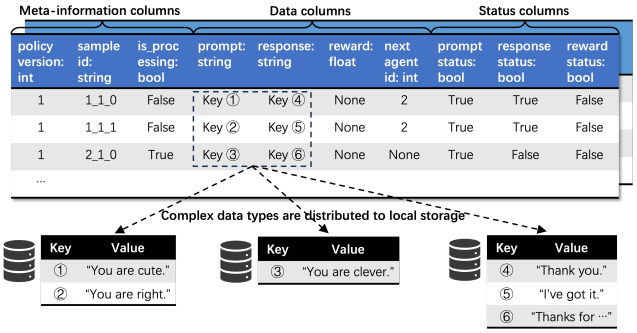


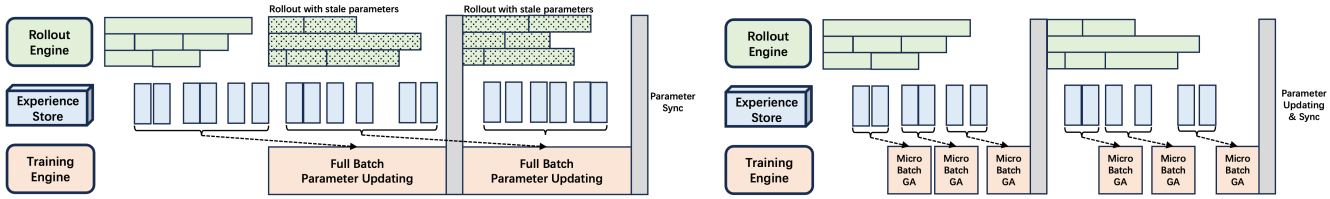
Figure 3: Multi-table structure of experience store.

Specifically, meta-information columns store essential metadata for each sample, including the policy version, sample id, and a processing flag indicating whether a sample has been read but not yet updated. The sample id is a semantically meaningful identifier of the form "`sample_id = {input_id}_{number_of_turns}_{trajectory_id}`". Combined with the `policy_version` field, this identifier ensures global uniqueness, deterministic ordering, and complete traceability across asynchronous MARL workflows. Data columns contain user-defined fields that store sample content, such as `prompt` and `response`. Each data column is paired with a boolean status column that specifies whether its corresponding value has been fully generated.

To support lightweight querying while accommodating extensible user-defined fields across MARL algorithms, the experience store is a type-aware hybrid storage module that automatically selects value-based or reference-based storage. All meta-information and status fields use simple data types (including int, float, and bool), which are stored by value for fast in-table access. Complex data types (including strings, lists, and tensors) are stored by reference, and the table only records the location key.

4.3 Fine-Grained Asynchronous Pipeline

As the foundation for rollout-training orchestration, the experience store provides a structured abstraction for fine-grained sample lifecycle management and flexible sample filtering. On this basis, we need to determine the collaborative paradigm between the two phases. Conventional synchronous paradigms introduce severe pipeline bubbles [8, 36, 44, 60]. The training resources remain idle during experience collection, and vice versa. To fully unlock the potential of disaggregation, we can exploit an asynchronous pipeline to overlap the rollout and training phases. As shown in Figure 4(a), existing training frameworks usually use one-step or fully asynchronous schemes to improve training throughput [7, 12, 58]. Although previous RL systems typically accept relaxed synchronization constraints, MARL is more sensitive to parameter staleness, making these asynchronous



(a) One-Step Asynchronous Pipeline

(b) Micro-Batch Asynchronous Pipeline

Figure 4: Different rollout-training asynchronous pipelines.

paradigms incompatible with multi-agent settings. If parameter updates from training are not propagated promptly, the agents produce responses using stale parameters. The cumulative bias gradually amplifies during multi-agent interactions, ultimately causing catastrophic damage to training stability and accuracy [8].

Therefore, FlexMARL employs a fine-grained asynchronous pipeline that overlaps the rollout and training stages while preserving the training semantics of synchronous on-policy RL. The key insight is that gradient accumulation (GA) across micro batches maintains mathematical equivalence with full batch updates. This allows us to decompose the global batch into several micro batches and restructure the pipeline, as illustrated in Figure 4(b). Policy training is triggered incrementally in a fine-grained manner without waiting for the entire batch (including long-tail trajectories). Concretely, the generated trajectories and their corresponding rewards are collected into the experience store. Thanks to the multi-table organization of the experience store, each agent updates independently based on its own table, enabling heterogeneous policy models and training configurations across agents. Once the accumulated experience exceeds a predefined micro-batch threshold, they are dispatched to the corresponding training process for gradient computation. This allows training to start much earlier and execute in parallel with the remaining rollouts, thereby hiding the tail latency of trajectory generation behind useful computations.

More importantly, we carefully manage policy versions by decoupling gradient computation from parameter updates. During micro batch training, each agent accumulates its computed gradients in its corresponding cache, and policy models are not updated immediately. After processing micro batches equivalent to the global batch, all agents aggregate the cached gradients to perform unified weight updating (policy version+1). The updated parameters are synchronized to all inference instances via high-bandwidth D2D interconnects, ensuring that the next rollout uses the up-to-date policy model. Leveraging the version-tracking capability of our experience store, FlexMARL ensures that trajectory generation always uses the most recent consistent policy snapshot, avoiding the parameter staleness problem common in traditional asynchronous pipelines. In summary, our asynchronous pipeline achieves a win-win situation for training efficiency and model accuracy. On the one hand, the training of early micro batches overlaps with the rollout of later micro batches, alleviating the synchronization barriers. On the other hand, incremental gradient caching and unified parameter updating help preserve training semantics. Unlike one-off pipelines, FlexMARL ensures that convergence behavior is logically consistent with synchronous training.

5 ROLLOUT ENGINE

5.1 Parallel Sampling

The processing of rollout requests typically follows a strictly sequential execution model, where the user query is processed through a sequence of agents to generate samples. Each agent may generate additional candidate samples for the same prompt. These samples are then further propagated through downstream agents to generate complete multi-agent trajectories. The next user query can be processed only after the entire rollout of the current query has finished. While this design preserves clear execution semantics, it severely limits hardware utilization and system throughput, especially in large-scale MARL settings.

To eliminate these inefficiencies, our elastic rollout engine restructures the execution pipeline into a fully parallelized, dependency-driven scheduling model. Once the upstream outputs of a rollout request are available, it can be dispatched to downstream agents immediately. Other queries or branches are independent of the completion state of the current query. Specifically, FlexMARL introduces multiple inference instances for two forms of parallelism:

- Inter-Query Parallelism: multiple queries are allowed to execute inference independently and in parallel.
- Intra-Query Parallelism: multiple trajectory generation for the same query can be executed in parallel.

This transformation converts the trajectory sampling from a query-level blocking model into a concurrent execution model. All rollout requests compete fairly for inference resources subject to data dependencies, thereby significantly improving rollout efficiency.

5.2 Hierarchical Load Balancing

Due to agents' functional specialization, MARL exhibits a skewed rollout load, resulting in persistent queuing delays at overloaded agents and underutilization at others. To mitigate load imbalance, our rollout engine provides a two-tier elastic scaling that balances load both within and across agents.

Intra-Agent Load Balancing. For each agent, we deploy multiple inference instances to handle its concurrent rollout request stream. A dedicated rollout manager employs a min-heap data structure to track the instantaneous load of backend inference instances. Upon receiving a new rollout request, the manager dispatches it to the instance with the lowest current load. This greedy scheduling scheme ensures balanced request distribution within inference instances of individual agents. Moreover, the rollout manager ensures fault tolerance by removing completed requests, canceling timed-out ones, and re-queuing unfinished ones.

Inter-Agent Load Balancing. While intra-agent balancing mitigates contention within a single agent, it cannot resolve resource starvation across agents with heterogeneous demands. As shown in Figure 5, the rollout manager polls the request queue lengths across all agents. We analyze these load metrics and then compute the load disparity between the highest and lowest load agents. When this disparity exceeds a configurable threshold Δ, a scaling operation is triggered to migrate inference capacity from underutilized agents to overloaded ones. Specifically, the number of migrated instances equals the difference in queue lengths, subject to the constraint that each agent retains at least one active inference instance to preserve liveness. The conservative policy prevents transient load oscillation while ensuring service availability for all agents. Migrating inference capacity requires rapid model weight transfer to newly allocated instances. FlexMARL abstracts multi-tier pathways into a unified and location-agnostic `Set`/`Get` API, which hides the complexities of D2D, H2D, and D2H communication. When a scaling operation is triggered, the agents requiring scaling up (Agent A) publish model weights via `Set` API, while the reallocated instances of scale-down agents (Agent B) overwrite their original weights via `Get` API. The implementation of `Set`/`Get` API will be detailed in §7. Finally, the migrated instance detaches from the donor Agent B and registers with the target Agent A to accept new rollout requests.

6 TRAINING ENGINE

6.1 Agent-Centric Resource Allocation

The fine-grained asynchronous pipeline incrementally triggers policy optimization. The agent enters the training phase when the joint orchestrator dispatches micro batches of samples from the experience store. As a result, the set of agents undergoing policy optimization and their entry is

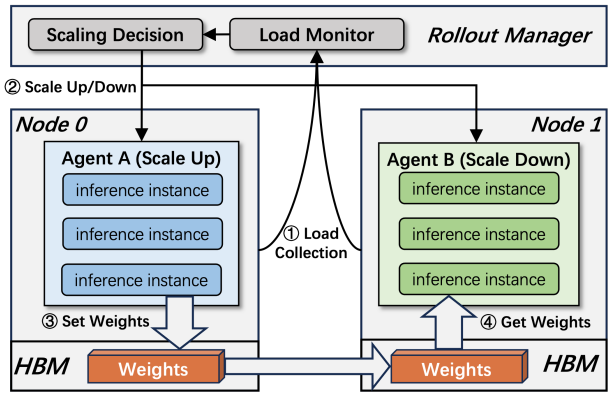


Figure 5: Inter-agent load balancing in rollout engine.

non-deterministic. Static allocation inevitably leads to significant resource waste, as inactive agents monopolize memory and computing resources throughout training. To accommodate such dynamic resource demands exhibited by LLM-based MARL, our training engine integrates agent-centric allocation to bind training resources only where and when needed. In particular, we implement time-division multiplexing for training resources via process group abstraction, which encapsulates all training processes associated with an individual agent. The training engine initializes a process group for each agent and obtains handles for all required processes. Such an abstraction activates, suspends, and resumes the training processes using a gang-scheduling strategy [6], enabling collective lifecycle management.

Once the experience store accumulates sufficient training samples for a specific agent, the corresponding process group activates the training processes and dynamically schedules them to available resources within the cluster. After an agent completes training for the current micro batch or when no new experiences are available, the process group is terminated. Unlike naive suspension, which retains process contexts in memory, our training engine terminates processes and immediately releases all associated hardware resources (including computing cores and HBM) back to the cluster pool. When the joint orchestrator detects new experiences for the agent, the process group is re-created on available resources, and policy training resumes from the last checkpoint. This "suspend-to-destroy" design ensures that storage and computation resources are never locked by inactive agents. In this way, the training resource allocation in FlexMARL is aligned with agent-specific workloads rather than being preassigned to fixed hardware, maintaining high training throughput even with a massive number of potential agents.

6.2 Training State Swap

With dynamic reallocation of hardware resources, training states (including weights and optimizer states) cannot remain

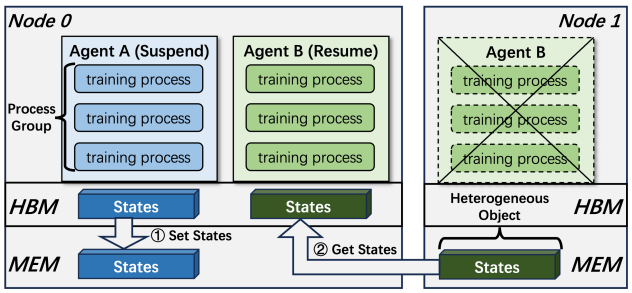


Figure 6: Efficient swap-in/out of training states for agent-centric resource allocation.

resident in HBM after process group suspension. Retaining the states for inactive agents would quickly exhaust device memory capacity and cause out-of-memory (OOM) failures, especially when the number of potential agents far exceeds available hardware resources. Consequently, on-demand resource binding necessitates efficient state swap between device and host memory. Herein, we again leverage `Set`/`Get` API (§7) to decouple logical state references from physical storage, enabling the training engine to track and access agent states consistently across distributed nodes.

As presented in Figure 6, the process group for the suspended agent (Agent A) triggers a checkpoint. Upon a `Set` operation, the current training states are offloaded from device memory to host memory as heterogeneous objects. For the resumed agent (Agent B), the training engine exploits locality-aware deployment to minimize state migration latency. The process group is prioritized for scheduling to its previously deployed node. Finally, the training engine restores the states into the destination device memory through the `Get` API and creates the corresponding process group.

7 SYSTEM IMPLEMENTATION

vLLM v0.9.1 is integrated as the rollout backend to ensure low-latency inference. Policy training is built on DeepSpeed v0.16.9 with ZeRO-3 enabled [30]. To relieve developers from the complexities of manual memory management and transmission, we introduce heterogeneous objects for data in device and host memory. Heterogeneous objects are encapsulated within a unified interface that follows key-value (KV) semantics. Then, FlexMARL provides `Set`/`Get` API supporting more general transfer operations, including intra-domain communication (D2D) and cross-tier communication (H2D/D2H). In particular, each node in the cluster maintains a dedicated resident daemon, which is responsible for managing the distributed metadata of heterogeneous objects. The host and device memory are *logically* unified to facilitate seamless data transfer across storage hierarchies.

D2D Transfer. The resident daemon utilizes a publish-subscribe coordination pattern where the invocation of the `Set` API triggers the registration of the device memory segment. This process captures critical location metadata, including physical device address, memory offset, and node-level identifiers. Data retrieval via the `Get` API initiates a multi-stage resolution process. The receiver queries the resident daemon to obtain the physical location and device ID of heterogeneous objects. Once the peer identity is established, the sender/receiver instantiates or reuses a specialized communication domain to execute point-to-point primitives, thereby completing the D2D data transfer.

H2D/D2H Transfer. Upon invoking the `Set` API, the sender instantiates a buffer in the host memory. The resident daemon then registers metadata that links the device memory extents to the corresponding host-side buffer. Following metadata registration, heterogeneous objects are asynchronously migrated from the device to the host memory via D2H transfer. When a receiver initiates a `Get` request, the location metadata can be queried from the resident daemon using the user-defined key. The heterogeneous objects are directly copied from the local host memory to the target device. In the case of cross-node retrieval, the resident daemon synchronizes the remote sender and the local receiver to facilitate a high-throughput, zero-copy migration via an RDMA-enabled network. Once the data is staged within the local host domain, it is finalized through a remote Host-to-Device (RH2D) operation.

8 PERFORMANCE EVALUATION

Our evaluation aims to answer the following six research questions (RQ):

- **RQ1**: How effective is FlexMARL in improving overall training performance against existing MARL frameworks?
- **RQ2**: How well does FlexMARL balance the rollout load across multiple agents?
- **RQ3**: Can FlexMARL improve the utilization of hardware resources?
- **RQ4**: What is the swap-in/out overhead of training states across different model sizes?
- **RQ5**: Do load balancing and asynchronous pipeline contribute to MARL training efficiency, respectively?
- **RQ6**: How does FlexMARL perform under large-scale deployments with increasing agent counts and model sizes?

8.1 Experimental Setup

Models and Datasets. We validate the effectiveness and scalability of FlexMARL on two real-world industrial datasets. For the Merchant Assistant (MA) dataset, multiple agents collaborate to help merchants perform complex store management tasks, including sales performance analysis, marketing

Table 2: Overall training performance of different MARL frameworks on the MA and CA datasets.

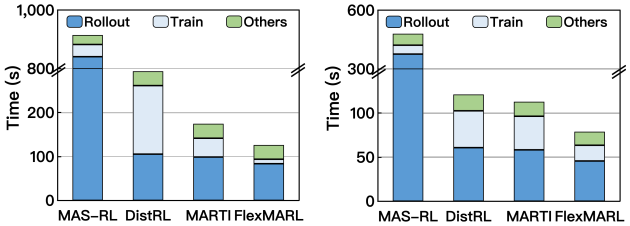
Dataset	Framework	E2E Time	Speedup	Throughput
MA	MAS-RL	914.4s	1.0×	119.0tps
	DistRL	293.8s	3.1×	401.0tps
	MARTI	174.1s	5.3×	642.8tps
	FlexMARL	126.1s	7.3×	910.2tps
CA	MAS-RL	438.6s	1.0×	265.5tps
	DistRL	130.0s	3.4×	571.6tps
	MARTI	112.8s	3.9×	655.9tps
	FlexMARL	78.8s	5.6×	821.4tps

strategy optimization, and after-sales service. We employ the Qwen2.5-14B model as the backbone, in which each agent does not share parameters and independently optimizes its policy. For the Category Assistant (CA) dataset, MAS is responsible for order querying, pricing strategies, and inventory management suggestions. The policy models involve two different sizes, i.e., Qwen2.5-14B and Qwen2.5-32B. These datasets are constructed for specific e-commerce tasks to simulate multi-agent collaboration requirements. Each sample comprises structured user queries, historical interaction context, and ground-truth agent responses. The detailed information about multi-agent assistant datasets is hidden due to business and confidentiality concerns.

Training Configurations. We conduct experiments on a 48-node cluster. Each node is equipped with 16 commercial NPUs (64GB memory) and interconnected via HCCS. At the foundational layer, the vendor-specific SDK and hardware abstraction library provide the necessary compute kernels, interfaced through a PyTorch adapter. For parallel sampling, inter-query and intra-query parallelism are set to 4 and 16, respectively, to accelerate the processing of user queries. The maximum length of response tokens is 8192. The load-balancing disparity threshold Δ is set to 5. We adopt the GRPO algorithm [31] for policy training and use a learning rate of 1e-6 with the Adam optimizer. The batch size and micro batch size in our fine-grained asynchronous pipeline are set to 64 and 16, respectively. All experiments are run with a fixed random seed of 2048 for reproducibility. Unless otherwise specified, all frameworks share the same hyperparameters to ensure fair evaluation.

Baselines. Most existing RL frameworks [7, 15, 32] are designed for a single model and are not suitable for MARL. Under such circumstances, we compare our proposed framework against the following MARL baselines:

- **MAS-RL** migrates the single-agent RL training framework to multi-agent settings. The naive MARL implementation adopts the colocated architecture and the same optimization pipeline as agentic RL to perform multi-agent collaborative rollout and training.
- **DistRL** enables flexible resource allocation through the disaggregated architecture. The rollout and training phases are deployed on dedicated resource pools, avoiding frequent onload/offload operations.
- **MARTI** (ICLR 2026) [50] is the state-of-the-art MARL framework that supports centralized multi-agent interactions and distributed policy training. Asynchronous rollouts are designed to further improve the training efficiency of MARL.



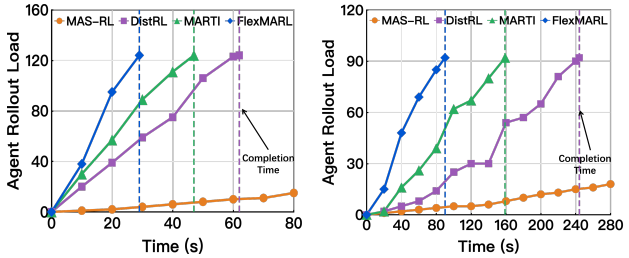
(a) MA Dataset (b) CA Dataset

Figure 7: E2E time breakdown of different MARL frameworks on the MA and CA datasets.

Evaluation Metrics. The design goal of FlexMARL is to facilitate efficient training of MARL algorithms on large-scale infrastructure. We will not modify the original algorithm logic, thus preserving model accuracy. Therefore, our experiments focus on training efficiency metrics to comprehensively evaluate performance. (1) E2E time records the average latency of all phases (rollout, training, and others) for a single sample. (2) Speedup is calculated as the ratio of the E2E time between frameworks and MAS-RL, quantifying the relative performance improvement. (3) Throughput is denoted by the number of generated tokens per second. (4) Agent load is reflected by the total number of rollout requests that each agent has processed. (5) Hardware utilization rate represents the percentage of time that AI cores remain active within a given training period.

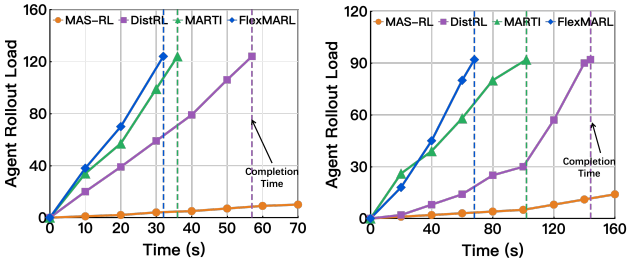
8.2 RQ1: Overall Performance

We first evaluate the overall performance of FlexMARL and three representative baselines. Table 2 summarizes the overall training efficiency, and Figure 7 shows the performance breakdown during rollout and training phases on the MA and CA datasets. FlexMARL consistently delivers the highest training speed and throughput. We highlight three observations from the experimental results. Firstly, our proposed framework outperforms MAS-RL by a large margin. For example, FlexMARL simultaneously reduces rollout and training time, ultimately achieving end-to-end speedups of 7.3× and 5.6× on the MA and CA datasets. The corresponding



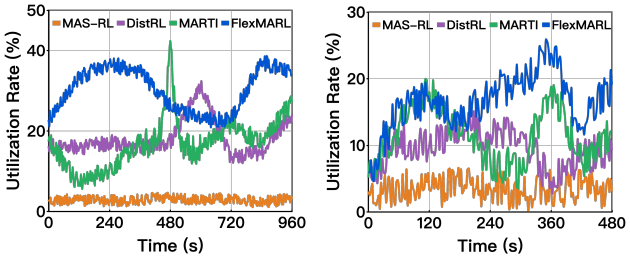
(a) Agent A (b) Agent B

Figure 8: Processed rollout load of representative agents on the MA dataset.



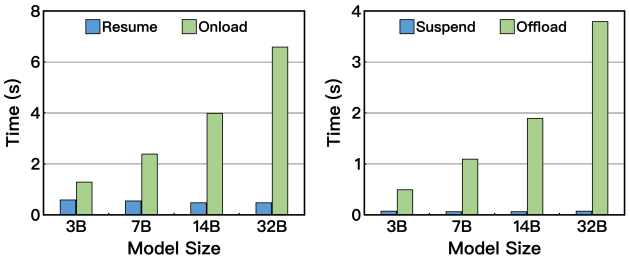
(a) Agent A (b) Agent B

Figure 9: Processed rollout load of representative agents on the CA dataset.



(a) MA Dataset (b) CA Dataset

Figure 10: Resource utilization rates of different MARL frameworks on the MA and CA datasets.



(a) Swap-in Overhead (b) Swap-out Overhead

Figure 11: Swap-in/out overhead of training states across different model sizes.

throughput improvements are 7.6× and 3.1×, respectively. The naive adaptation in MAS-RL cannot address MARL-specific challenges and suffers from severe queuing delays. FlexMARL leverages parallel sampling and elastic scaling to eliminate such bottlenecks, reducing rollout latency by 86% on average. Secondly, even with the disaggregated architecture, the E2E speed of DistRL is only 42%–61% of ours. This is because DistRL relieves resource contention but introduces significant pipeline bubbles, resulting in resource waste. Our fine-grained asynchronous pipeline overlaps rollout and training phases, hiding tail latency. Compared to DistRL, FlexMARL reduces policy training time from 155.9s to 10.2s on the MA dataset. Thirdly, FlexMARL improves training speed and throughput by 1.4× and 1.3× over the state-of-the-art MARTI framework on average. Although asynchronous rollouts can speed up experience collection, their fixed resource allocation strategy fails to accommodate skewed and dynamic workloads. The results above demonstrate that our co-design of the rollout, training, and orchestration phases provides substantial performance gains.

8.3 RQ2: Agent Rollout Load

To validate the rollout efficiency, we select two representative agents from two datasets and monitor the number of processed requests within a single step. Figures 8 and 9 plot the load patterns of different frameworks. We find that FlexMARL processes rollout requests much faster than the baselines for all datasets. For instance, FlexMARL completes all pending requests within 90s for Agent B on the MA dataset, while DistRL and MARTI require about 244s and 159s. The corresponding speedups are 2.7× and 1.8×, respectively. In particular, MAS-RL maintains a constant workload throughout training, as it processes all queries serially without parallelization, resulting in the longest completion time. Another interesting observation is that the queued requests of FlexMARL exhibit a distinct pattern, i.e., faster accumulation followed by quicker reduction compared to other methods. The explanation for this phenomenon is that upstream agents also leverage load balancing to efficiently process their tasks, thereby accelerating request propagation along the multi-agent workflow. In contrast, the baselines cannot adjust the number of inference instances. The core agents experience persistent queueing of rollout requests, which blocks the MARL training process. These results confirm the flexibility and superiority of hierarchical load balancing.

8.4 RQ3: Resource Utilization

We measure the overall hardware utilization of FlexMARL and three baselines throughout the MARL training process. The results on the MA and CA datasets are presented in Figure 10. We observe that FlexMARL maintains higher resource utilization rates compared to baselines, achieving an average

Table 3: Ablation study of FlexMARL on the MA and CA datasets.

Dataset	Variant	E2E Time	Speedup	Throughput
MA	w/o balancing	152.2s	6.0×	729.9tps
	w/o async	256.2s	3.6×	444.0tps
	FlexMARL	126.1s	7.3×	910.2tps
CA	w/o balancing	95.9s	4.6×	611.8tps
	w/o async	124.1s	3.5×	608.4tps
	FlexMARL	78.8s	5.6×	821.4tps

Table 4: Performance evaluation of FlexMARL in large-scale heterogeneous deployments.

Configuration	Rollout	Training	E2E Time	Throughput
5×32B	96.2s	54.5s	160.3s	265.9tps
3×32B + 7×14B	96.7s	15.6s	132.5s	334.8tps
15×14B	35.4s	5.3s	41.9s	754.2tps

of 32.4% on MA and 19.8% on CA. Other frameworks struggle with inefficient resource usage. For example, MAS-RL and MARTI exhibit severe utilization gaps with an average of only 3.6% and 12.3% on the CA dataset, respectively. This inefficiency stems from the colocated architecture that forces training and rollout to use the same number of resources. Heterogeneous requirements of different phases result in inherent resource conflict. Moreover, DistRL shows noticeable utilization drops, i.e., 16.9% on MA and 10.2% on CA. The rollout and training phases are executed alternately, leaving training resources idle during long-tail trajectory generation. FlexMARL dynamically binds resources to agents on demand through agent-centric resource allocation, thereby improving utilization. The above results imply that our proposed framework can effectively eliminate resource idleness and leverage hardware capabilities to enhance training efficiency.

8.5 RQ4: State Swap Overhead

To assess the practicality of the training engine, we record the state swap overhead across four representative model sizes. The swap-in overhead includes resuming the process group and loading states back into device memory. The swap-out overhead includes suspending the process group and offloading training states from the device to host memory. As shown in Figure 11, both suspend and resume latencies are minimal and stay nearly constant regardless of model scale, indicating that our process group abstraction causes negligible control-plane cost. Besides, the onload and offload overhead grow gradually with model size. For example, offload latency increases from 0.5s to 3.8s with model size varying from 3B to 32B. Nevertheless, our state swap overhead is only 11s for the largest model, which can be effectively hidden behind rollout latency. These results demonstrate that our training engine enables flexible agent-centric resource allocation without compromising MARL efficiency.

8.6 RQ5: Ablation Study

We conduct ablation studies to quantify the individual contributions of hierarchical load balancing and micro-batch asynchronous pipeline. Performance metrics are evaluated against the full FlexMARL framework, with results summarized in Table 3. Disabling hierarchical load balancing reduces the training throughput by 19.8%-25.5%, and the speedup drops from 7.3× to 6.0× for the MA dataset. Without load balancing to elastically reallocate inference capacity, the rollout phase will be bottlenecked by overloaded agents, slowing down the entire training process. For the variant without the micro-batch asynchronous pipeline, the performance degradation is even more pronounced. The E2E time increases by 103.2% for the MA dataset and 57.5% for the CA dataset. This degradation stems from severe pipeline bubbles, in which training resources remain idle while waiting for straggler agents. Our FlexMARL framework reverts to synchronous paradigms that cannot hide long-tail latency. The above results highlight the importance of load balancing and asynchronous pipeline for MARL training efficiency.

8.7 RQ6: Scalability

To evaluate the scalability of FlexMARL, we investigate system performance under large-scale heterogeneous deployments. Table 4 reports the E2E time and training throughput on the MA dataset across varying agent counts and model sizes. FlexMARL demonstrates robust scalability, achieving a peak throughput of 754.2tps with 15 agents. Critically, FlexMARL successfully supports complex heterogeneous configurations, such as a mixed multi-agent setting of 3×32B and 7×14B. In contrast, existing MARL frameworks (including MARTI) lack the cross-node resource placement capabilities required for such heavy, uneven workloads, which typically results in OOM errors. Our end-to-end optimizations enable FlexMARL to maintain high throughput even when involving multiple 32B agents, demonstrating its unique capability to support industrial-scale MARL workloads.

9 DISCUSSION

Building FlexMARL and deploying it on a production cluster provided unique insights into the LLM-based MARL training. We summarize our experiences into the following key lessons during MARL system development.

Hardware-Aware Abstraction. Modern distributed frameworks usually abstract away physical device details to simplify development. However, the absence of stable UUIDs

in the NPU driver stack led to failures in high-level zero-copy mechanisms. FlexMARL addresses this by enforcing process-level determinism and mapping internal PIDs to logical ranks. Furthermore, we find that existing fine-grained, parameter-by-parameter synchronization schemes are costly. Specifically, control-plane overhead, primarily driven by the task scheduling and kernel launching, accounts for over 99% of synchronization latency when iterating over billions of parameters. This suggests that the cost of invoking communication primitives could dwarf the actual time spent on transferring data. FlexMARL aggregates weights into a single contiguous memory buffer, reducing synchronization complexity from $O(N_{params})$ to $O(1)$ and achieving a 200× speedup. These experiences reveal that system software for accelerators must aggressively batch control-plane operations to amortize scheduling costs and enforce explicit addressing to bypass unstable driver abstractions.

Cross-Node Agent Deployment. Agent deployment in existing frameworks (e.g., MARTI) fails to scale effectively across multiple nodes, revealing critical performance bottlenecks. The reason is that all resources across nodes are managed via a single placement group (PG) using a "PACK" scheduling strategy[1]. Both training and inference processes are deployed in the same PG. By generating a bundle list derived from a predefined device set, each process is bound to its corresponding device bundle. However, since the logical bundle ordering is not equivalent to physical device IDs, the "PACK" strategy fails in multi-agent settings. Training/inference processes of the same agent may be scheduled to different nodes, leading to significant communication overhead and potential network failure risk [26]. By shifting scheduling granularity from a cluster level to individual nodes, our framework instantiates an independent PG for each node using a "STRICT_PACK" strategy[1]. We establish a deterministic one-to-one mapping between logical bundles and physical device IDs, which effectively eliminates unintended cross-node communication and enhances system stability during MARL deployments.

10 RELATED WORK

Frameworks for LLM Post-Training. RL has emerged as a critical paradigm for enhancing LLM capabilities [10, 48], and it relies heavily on scalable training infrastructure [9, 25, 55]. Early frameworks like ColossalChat [19] and DeepSpeed-Chat [45] adopt the colocated architecture where the rollout and training phases share the same resource pool, leading to serialized execution and resource contention. To improve throughput, recent frameworks such as OpenRLHF [15], veRL [32], and AReaL [7] have employed the disaggregated architecture that allocates distinct resource pools to rollout and training phases. They enable asynchronous pipeline and leverage distributed runtimes [5, 26] to orchestrate the different phases, thereby achieving impressive training efficiency. Building upon these foundations, AgentRL [49] and SkyRL-Agent [3] further extend a fully-asynchronous rollout-training pipeline to support efficient, multi-turn, long-horizon agentic RL training. However, these solutions are primarily designed for a single model and cannot accommodate the imbalanced workload across different agents. Once directly applied to MARL settings, it typically results in long-tail rollout latency. Besides, general-purpose RL frameworks overlook how to allocate resources to multiple agents, which causes resource underutilization. In contrast to general-purpose RL frameworks, FlexMARL co-designs the rollout, training, and their orchestration, tailored to the complex and dynamic nature of large-scale MARL.

Frameworks for Multi-Agent Systems. Prior works either target MARL on structured environments [2] rather than LLM-based agents in social simulation [1], web tasks [54], and recommendation systems [27], or algorithm-level optimization on communication protocols [53], coordination strategies [11, 29, 37], and reward functions [33, 40]. Regarding system-level works, some libraries (such as AutoGen [43], MetaGPT [14], and Camel [18]) simplify deployment through high-level abstractions [20] without collaborative multi-agent training support [22, 34, 46]. The MARLess [39] framework remains limited to CPU clusters and conventional neural network-based MARL. Other MARL frameworks [50, 56] only provide basic training capabilities for LLM-based MARL and treat the underlying networked infrastructure as a black box [17]. These solutions typically employ the synchronous rollout-training paradigm and static resource allocation for different agents, leading to synchronization barriers and resource underutilization, especially when multi-agent workloads are imbalanced [4]. The above gaps with systematic challenges are fulfilled by FlexMARL as shown in Table 1 compared with state-of-the-art baselines.

11 CONCLUSION

In this paper, we present FlexMARL, an end-to-end rollout-training co-designed framework for large-scale LLM-based MARL. FlexMARL bridges the gap between MARL algorithms and networked infrastructure by introducing three key innovations: parallel sampling with hierarchical load balancing, on-demand resource binding for training, and a fine-grained asynchronous pipeline with strong consistency. These system-level optimizations effectively balance multi-agent load and maximize resource utilization. The evaluation results on a production cluster confirm the superior scalability and efficiency of our proposed framework.

[1]https://docs.ray.io/en/latest/ray-core/scheduling/placement-group.html

REFERENCES

[1] Jinyu Cai, Jialong Li, Mingyue Zhang, Munan Li, Chen-Shu Wang, and Kenji Tei. 2024. Language evolution for evading social media regulation via llm-based multi-agent simulation. In *2024 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 1–10.

[2] Lorenzo Canese, Gian Carlo Cardarilli, Luca Di Nunzio, Rocco Fazzolari, Daniele Giardino, Marco Re, and Sergio Spanò. 2021. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences* 11, 11 (2021), 4948.

[3] Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, et al. 2025. Skyrl-agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108* (2025).

[4] Guanzhong Chen, Shaoxiong Yang, Chao Li, Wei Liu, Jian Luan, and Zenglin Xu. 2025. Heterogeneous Group-Based Reinforcement Learning for LLM-based Multi-Agent Systems. *arXiv preprint arXiv:2506.02718* (2025).

[5] Qiong Chen, Jianmin Qian, Yulin Che, Ziqi Lin, Jianfeng Wang, Jie Zhou, Licheng Song, Yi Liang, Jie Wu, Wei Zheng, et al. 2024. Yuanrong: A production general-purpose serverless system for distributed applications in the cloud. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 843–859.

[6] Dror G Feitelson and Morris A Jettee. 1997. Improved utilization and responsiveness with gang scheduling. In *Workshop on Job Scheduling Strategies for Parallel Processing*. Springer, 238–261.

[7] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. *arXiv preprint arXiv:2505.24298* (2025).

[8] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, et al. 2025. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training. *arXiv preprint arXiv:2509.21009* (2025).

[9] Hao Ge, Junda Feng, Qi Huang, Fangcheng Fu, Xiaonan Nie, Lei Zuo, Haibin Lin, Bin Cui, and Xin Liu. 2025. ByteScale: Communication-Efficient Scaling of LLM Training with a 2048K Context Length on 16384 GPUs. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 963–978.

[10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[11] Ai Han, Junxing Hu, Pu Wei, Zhiqian Zhang, Yuhang Guo, Jiawei Lu, and Zicheng Zhang. 2025. JoyAgents-R1: Joint Evolution Dynamics for Versatile Multi-LLM Agents with Reinforcement Learning. *arXiv preprint arXiv:2506.19846* (2025).

[12] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, et al. 2025. AsyncFlow: An Asynchronous Streaming RL Framework for Efficient LLM Post-Training. *arXiv preprint arXiv:2507.01663* (2025).

[13] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. 2025. History rhymes: Accelerating llm reinforcement learning with rhymerl. *arXiv preprint arXiv:2508.18588* (2025).

[14] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*.

[15] Jian Hu, Xibin Wu, Zilin Zhu, Weixun Wang, Dehao Zhang, Yu Cao, et al. 2024. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143* (2024).

[16] Tianyi Hu, Zhiqiang Pu, Xiaolin Ai, Tenghai Qiu, and Jianqiang Yi. 2024. Measuring Policy Distance for Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2401.11257* (2024).

[17] Ziqi Jia, Junjie Li, Xiaoyang Qu, and Jianzong Wang. 2025. Enhancing Multi-Agent Systems via Reinforcement Learning with LLM-based Planner and Graph-based Policy. *arXiv preprint arXiv:2503.10049* (2025).

[18] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems* 36 (2023), 51991–52008.

[19] Shenggui Li, Hongxin Liu, Zhengda Bian, Jiarui Fang, Haichen Huang, Yuliang Liu, Boxiang Wang, and Yang You. 2023. Colossal-ai: A unified deep learning system for large-scale parallel training. In *Proceedings of the 52nd International Conference on Parallel Processing*. 766–775.

[20] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* 1, 1 (2024), 9.

[21] Zhenghao Li, Zhi Zheng, Wei Chen, Jielun Zhao, Yong Chen, Tong Xu, and Enhong Chen. 2026. DynaDebate: Breaking Homogeneity in Multi-Agent Debate with Dynamic Path Generation. *arXiv preprint arXiv:2601.05746* (2026).

[22] Junwei Liao, Muning Wen, Jun Wang, and Weinan Zhang. 2025. Marft: Multi-agent reinforcement fine-tuning. *arXiv preprint arXiv:2504.16129* (2025).

[23] Shuo Liu, Tianle Chen, Zeyu Liang, Xueguang Lyu, and Christopher Amato. 2025. Llm collaboration with multi-agent reinforcement learning. *arXiv preprint arXiv:2508.04652* (2025).

[24] Francisco Martinez-Gil, Miguel Lozano, and Fernando Fernández. 2014. MARL-Ped: A multi-agent reinforcement learning based framework to simulate pedestrian groups. *Simulation Modelling Practice and Theory* 47 (2014), 259–275.

[25] Qingkai Meng, Hao Zheng, Zhenhui Zhang, ChonLam Lao, Chengyuan Huang, Baojia Li, Ziyuan Zhu, Hao Lu, Weizhen Dang, Zitong Lin, et al. 2025. Astral: A datacenter infrastructure for large language model training at scale. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 609–625.

[26] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.

[27] Guangtao Nie, Rong Zhi, Xiaofan Yan, Yufan Du, Xiangyang Zhang, Jianwei Chen, Mi Zhou, Hongshen Chen, Tianhao Li, Ziguang Cheng, et al. 2024. A hybrid multi-agent conversational recommender system with LLM and search engine in e-commerce. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 745–747.

[28] Ruwei Pan, Hongyu Zhang, and Chao Liu. 2025. CodeCoR: An LLM-based self-reflective multi-agent framework for code generation. *arXiv preprint arXiv:2501.07811* (2025).

[29] Chuhao Qin and Evangelos Pournaras. 2025. Strategic Coordination for Evolving Multi-agent Systems: A Hierarchical Reinforcement and Collective Learning Approach. *arXiv preprint arXiv:2509.18088* (2025).

[30] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

[32] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1279–1297.

[33] Haoran Su, Yandong Sun, and Congjia Yu. 2026. The End of Reward Engineering: How LLMs Are Redefining Multi-Agent Coordination. *arXiv preprint arXiv:2601.08237* (2026).

[34] Chuanneng Sun, Songjun Huang, and Dario Pompili. 2024. Llm-based multi-agent reinforcement learning: Current and future directions. *arXiv preprint arXiv:2405.11106* (2024).

[35] Xinchen Wan, Xinyu Yang, Kaiqiang Xu, Xudong Liao, Yilun Jin, Yijun Sun, Zhenghang Ren, Han Tian, and Kai Chen. 2025. Coflow Scheduling for LLM Training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 1232–1234.

[36] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, et al. 2025. Reinforcement Learning Optimization for Large-Scale Learning: An Efficient and User-Friendly Scaling Library. *arXiv preprint arXiv:2506.06122* (2025).

[37] Xin Wang, Chen Zhao, Tingwen Huang, Prasun Chakrabarti, and Jürgen Kurths. 2023. Cooperative learning of multi-agent systems via reinforcement learning. *IEEE Transactions on Signal and Information Processing over Networks* 9 (2023), 13–23.

[38] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, et al. 2025. Intent-driven network management with multi-agent LLMs: The confucius framework. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 347–362.

[39] Rui Wei, Hanfei Yu, Xikang Song, Jian Li, Devesh Tiwari, Ying Mao, and Hao Wang. 2025. Multi-Agent Reinforcement Learning with Serverless Computing. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 225–239.

[40] Yuan Wei, Xiaohan Shan, Ran Miao, and Jianmin Li. 2025. Lero: Llm-driven evolutionary framework with hybrid rewards and enhanced observation for multi-agent reinforcement learning. In *International Conference on Intelligent Computing*. Springer, 15–26.

[41] Christoph Wittner. 2026. Communication Methods in Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2601.12886* (2026).

[42] Duo Wu, Xianda Wang, Yaqi Qiao, Zhi Wang, Junchen Jiang, Shuguang Cui, and Fangxin Wang. 2024. Netllm: Adapting large language models for networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 661–678.

[43] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.

[44] LLM Xiaomi, Bingquan Xia, Bowen Shen, Dawei Zhu, Di Zhang, Gang Wang, Hailin Zhang, Huaqiu Liu, Jiebao Xiao, Jinhao Dong, et al. 2025. MiMo: Unlocking the Reasoning Potential of Language Model–From Pretraining to Posttraining. *arXiv preprint arXiv:2505.07608* (2025).

[45] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, et al. 2023. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales. *arXiv preprint arXiv:2308.01320* (2023).

[46] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. 2022. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in neural information processing systems* 35 (2022), 24611–24624.

[47] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.

[48] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. 2025. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476* (2025).

[49] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, et al. 2025. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework. *arXiv preprint arXiv:2510.04206* (2025).

[50] Kaiyan Zhang, Runze Liu, Xuekai Zhu, Kai Tian, Sihang Zeng, Guoli Jia, Yuchen Fan, Xingtai Lv, Yuxin Zuo, Che Jiang, Ziyang Liu, Jianyu Wang, Yuru Wang, Ruotong Zhao, Ermo Hua, Yibo Wang, Shijie Wang, Junqi Gao, Xinwei Long, Youbang Sun, Zhiyuan Ma, Ganqu Cui, Lei Bai, Ning Ding, Biqing Qi, and Bowen Zhou. 2025. MARTI: A Framework for Multi-Agent LLM Systems Reinforced Training and Inference. (2025). https://github.com/TsinghuaC3I/MARTI

[51] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. 2021. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of reinforcement learning and control* (2021), 321–384.

[52] Shaowei Zhang and Deyi Xiong. 2025. Debate4MATH: Multi-agent debate for fine-grained reasoning in math. In *Findings of the Association for Computational Linguistics: ACL 2025*. 16810–16824.

[53] Xinren Zhang, Jiadong Yu, and Zixin Zhong. 2025. Learning Efficient Communication Protocols for Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2511.09171* (2025).

[54] Yao Zhang, Zijian Ma, Yunpu Ma, Zhen Han, Yu Wu, and Volker Tresp. 2025. Webpilot: A versatile and autonomous multi-agent system for web task execution with strategic exploration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 23378–23386.

[55] Zili Zhang, Yinmin Zhong, Yimin Jiang, Hanpeng Hu, Jianjian Sun, Zheng Ge, Yibo Zhu, Daxin Jiang, and Xin Jin. 2025. Disttrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 24–38.

[56] Yujie Zhao, Lanxiang Hu, Yang Wang, Minmin Hou, Hao Zhang, Ke Ding, and Jishen Zhao. 2025. Stronger-MAS: Multi-Agent Reinforcement Learning for Collaborative LLMs. *arXiv preprint arXiv:2510.11062* (2025).

[57] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. 2025. Mage: A multi-agent engine for automated rtl code generation. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.

[58] Yinmin Zhong, Zili Zhang, Xiaoniu Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, et al. 2025. StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated Stream Generation. *arXiv preprint arXiv:2504.15930* (2025).

[59] Guobin Zhu, Rui Zhou, Wenkang Ji, and Shiyu Zhao. 2025. Lamarl: Llm-aided multi-agent reinforcement learning for cooperative policy generation. *IEEE Robotics and Automation Letters* (2025).

[60] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. 2025. Megascale-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 592–608.